

Exception handling and UnitTests

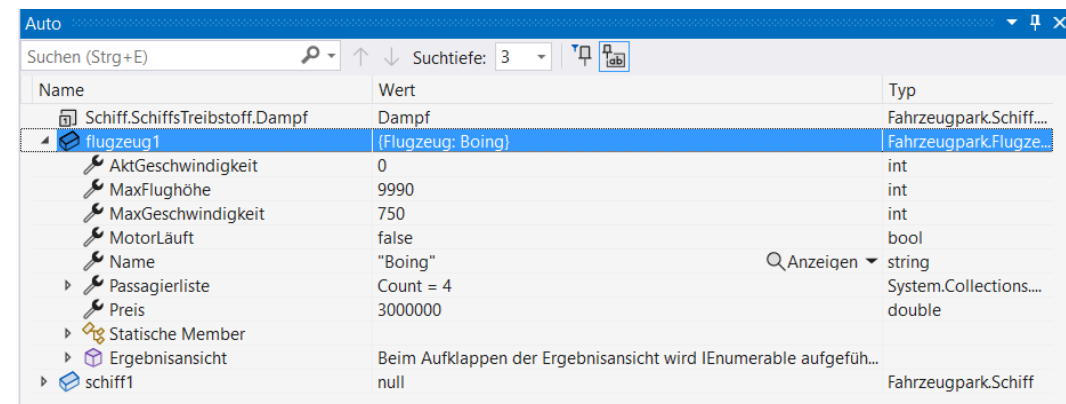
Debugging

- Breakpoints pause execution BEFORE the selected line is executed.

```
114 | | | Flugzeug flugzeug1 = new Flugzeug("Boing", 750, 3000000, 9990);  
115 | | | Schiff schiff1 = new Schiff("Titanic", 40, 3500000, Schiff.SchiffsTreibstoff.Dampf);  
116 | | |  
117 | | | BeladeFahrzeuge(pkwl, flugzeug1);
```

- Single step (F11): Stepwise execution with jump to method definitions
- Procedure step (F10): Stepwise execution on current level

- Auto and local windows contain current variable assignments



Name	Wert	Typ
Schiff.SchiffsTreibstoff.Dampf	Dampf	Fahrzeugpark.Schiff...
flugzeug1	{Flugzeug: Boing}	Fahrzeugpark.Flugze...
AktGeschwindigkeit	0	int
MaxFlughöhe	9990	int
MaxGeschwindigkeit	750	int
MotorLäuft	false	bool
Name	"Boing"	string
Passagierliste	Count = 4	System.Collections....
Preis	3000000	double
Statische Member		
Ergebnisansicht	Beim Aufklappen der Ergebnisansicht wird IEnumerable aufgeföh...	
schiff1	null	Fahrzeugpark.Schiff

Exception handling

- with a try-catch block errors can be caught and handled
- try block
 - actual application
- catch block
 - errors are caught and handled
- finally block
 - is optional
 - is guaranteed to be executed

```
try
{
    int input = int.Parse(Console.ReadLine());
}
catch (FormatException e)
{
    Console.WriteLine("Please enter a number");
    return;
}
catch (Exception e)
{
    Console.WriteLine(e.Message);
    return;
}
finally
{
    Console.WriteLine("Thank you");
}
```

create a custom exception

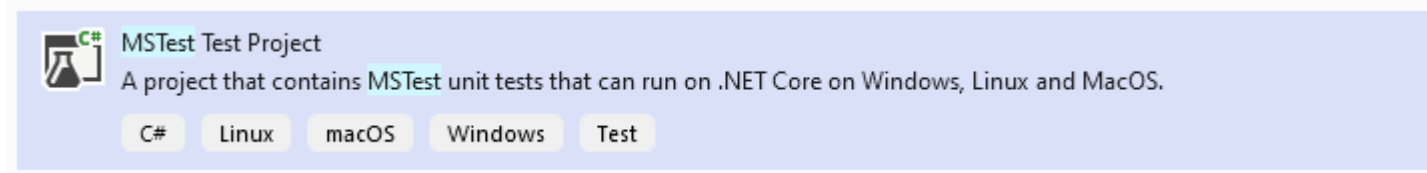
- each new exception is derived from the "Exception" class
- by "catch (Exception)" thus all not before caught error messages are handled

```
try
{
    throw new MyException("An exception has been thrown!");
}
catch (MyException e)
{
    Console.WriteLine(e.Message);
}
catch (Exception e)
{
    Console.WriteLine(e.Message);
}

public class MyException : Exception
{
    public MyException(string message) : base(message)
    {
    }
}
```

UnitTest

- with a UnitTest functions can be tested for a defined input and an expected return
- an extra project, which can be created in Visual Studio



Create a test class

- a class must be provided with the attribute "[TestClass]", so that VS recognizes it as test class
- a method must be provided with the attribute "[TestMethod]", so that VS recognizes it as a test method and calls it during a UnitTest

```
[TestClass]
0 references
public class UnitTest1
{
    [TestMethod]
    ✓ | 0 references
    public void TestMethod1()
    {
        ClassToTest ctt = new ClassToTest();
        for (int i = 0; i < 100; i++)
        {
            int half = ctt.Half(i);
            Assert.AreEqual(i / 2, half);
        }
    }
}
```

Run UnitTest

- to start a unit test click on the "Run all tests" item in the "Test" menu
- alternatively by using the key combination [CTRL]+[R]
[CTRL]+[A].
- in the test explorer the results are then displayed

